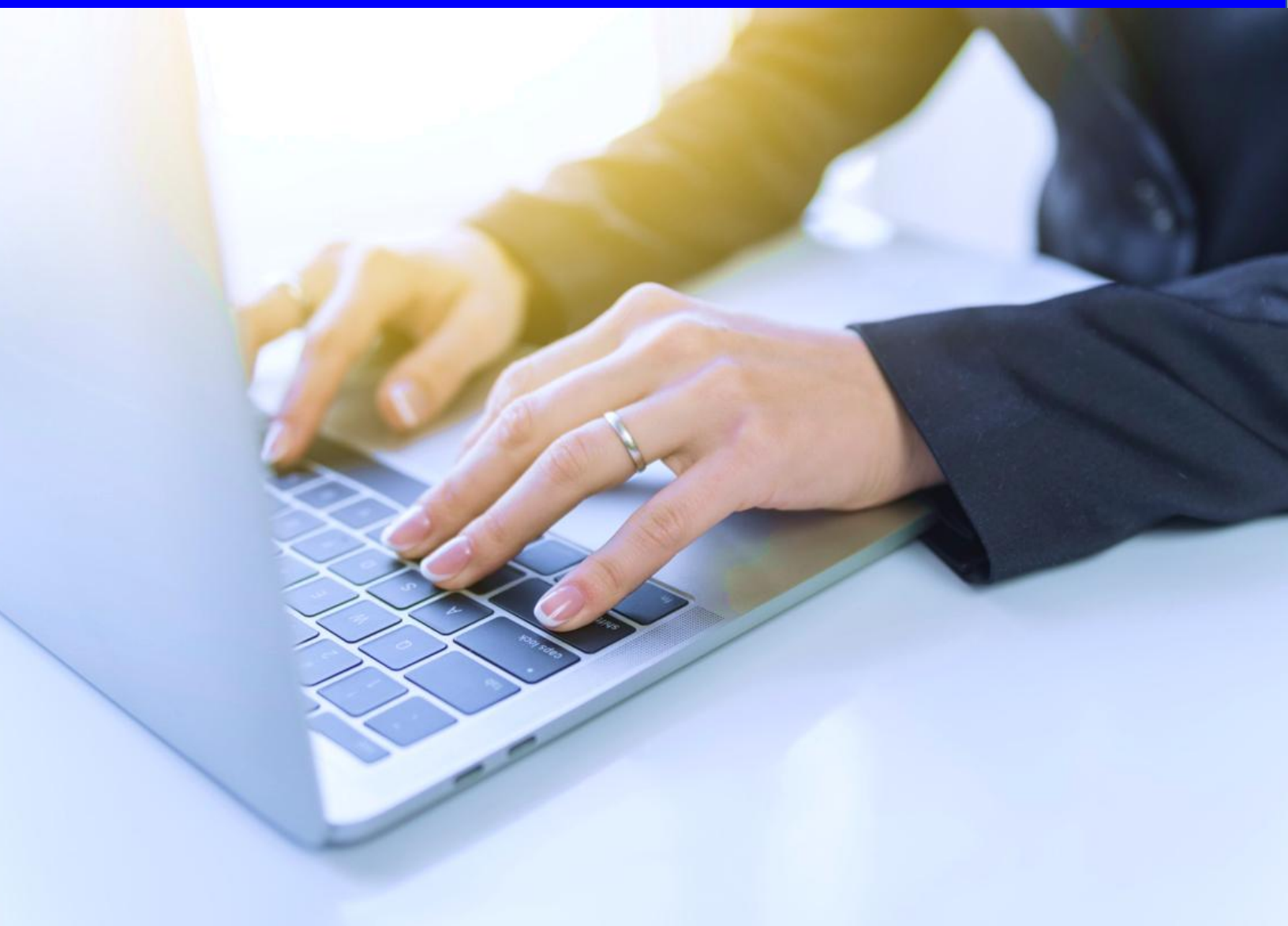


|                          |      |                                |
|--------------------------|------|--------------------------------|
| Nombre de la Asignatura: |      | CONSULTA DE DATOS              |
| Unidad                   | Nº 1 | CONSTRUYENDO CONSULTAS SIMPLES |



# Consideraciones Previas

El contenido que se expone a continuación está ligado a los siguientes aprendizajes esperados:

- Organiza la lógica de negocio considerando restricciones del lenguaje, requisitos de la lógica de negocios, requisitos de información y sistemas de gestión de base de datos para solucionar los requerimientos de información planteados.
- Utiliza operadores lógicos, de comparación, matemáticos y cláusulas de ordenamiento para solucionar los requerimientos de información planteados.

## SOBRE LAS FUENTES UTILIZADAS EN EL MATERIAL

*El presente Material de Estudio constituye un ejercicio de recopilación de distintas fuentes, cuyas referencias bibliográficas estarán debidamente señaladas al final del documento. Este material, en ningún caso pretende asumir como propia la autoría de las ideas planteadas. La información que se incorpora tiene como única finalidad el apoyo para el desarrollo de los contenidos de la unidad correspondiente, respetando los derechos de autor ligados a las ideas e información seleccionada para los fines específicos de cada asignatura.*

# Introducción

En el siguiente Material de Estudio, se conocerán los operadores lógicos como el AND, OR y NOT.

Además de las sentencias INSERT, que permiten añadir nuevas filas a una tabla. La sentencia UPDATE, que permite la modificación o actualización de datos dentro de una tabla. Y el comando DELETE, que elimina filas de una tabla de acuerdo con una condición o incluso todos los registros de ella.

Para ello, se utilizarán varios ejemplos, detallando la lógica y resultado esperado.

Se invita al estudiante a seguir leyendo e interiorizarse de las posibilidades que ofrecen las consultas a las bases de datos.

## Ideas fuerza

- Operadores lógicos: Los operadores lógicos son otro que podemos incorporar en las consultas de bases de datos. Se abordarán los operadores: AND, OR y NOT. Esto permitirá unir dos o más condiciones al momento de filtrar con el comando WHERE dentro de un SELECT.
- Sentencia INSERT: Sentencia que permite agregar nuevas filas a una tabla existente.
- Sentencia UPDATE: La instrucción UPDATE puede modificar uno o varios registros. Para actualizar todos los registros de una tabla, especifique el nombre de la tabla y, después, use la cláusula SET para especificar el campo o los campos que se desean modificar.
- Sentencia DELETE: La sentencia DELETE se utilizará cuando se desee eliminar ciertos registros de acuerdo con una condición o vaciar por completo una tabla.
- Sentencia TRUNCATE: La sentencia TRUNCATE se utilizará para eliminar todas las filas de una tabla. Esto tiene el mismo efecto que el comando DELETE, pero es más rápido que el comando DELETE para tablas grandes.

# Índice

|                                               |           |
|-----------------------------------------------|-----------|
| <i>Introducción.....</i>                      | <i>3</i>  |
| <i>Ideas fuerza.....</i>                      | <i>4</i>  |
| <i>Desarrollo.....</i>                        | <i>6</i>  |
| Operadores Lógicos .....                      | 6         |
| Operador AND .....                            | 7         |
| Operador OR.....                              | 8         |
| Operador NOT .....                            | 9         |
| Reglas de precedencia para Operadores.....    | 9         |
| Lenguaje de Manipulación de Datos (DML) ..... | 12        |
| Sentencia INSERT .....                        | 13        |
| .....                                         | 15        |
| Objeto Secuencia .....                        | 16        |
| Uso de Objeto Secuencia.....                  | 18        |
| Sentencia UPDATE .....                        | 19        |
| Sentencia DELETE .....                        | 21        |
| Sentencia TRUNCATE.....                       | 23        |
| <i>Conclusión.....</i>                        | <i>24</i> |
| <i>Referencias .....</i>                      | <i>25</i> |

# Desarrollo

## Operadores Lógicos

Cuando se requiere que una sentencia SQL evalúe más de una condición para visualizar, eliminar o actualizar, se requiere usar Operadores Lógicos en la cláusula WHERE.

Una condición lógica combina el resultado de dos condiciones para producir un único resultado basado en aquellas condiciones. Retorna una fila sólo si el resultado total de la condición es TRUE (verdadero).

Los operadores lógicos disponibles en SQL son: AND, OR y NOT

- AND: retorna verdadero si todas las condiciones que se evalúan son verdaderas.
- OR: retorna verdadero si alguna de las condiciones que se evalúan son verdaderas.
- NOT: retorna verdadero si la condición es falsa. Invierte la lógica de la expresión que está a su derecha. Si era verdadera, mediante NOT pasa a ser falso.

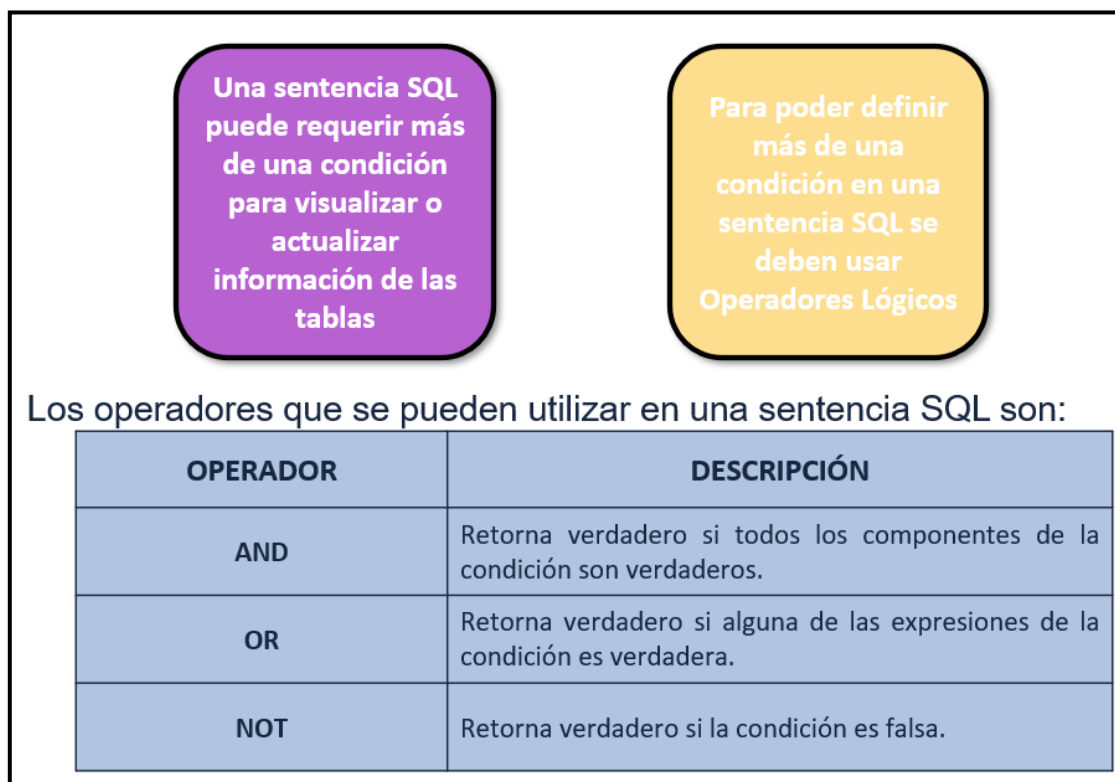


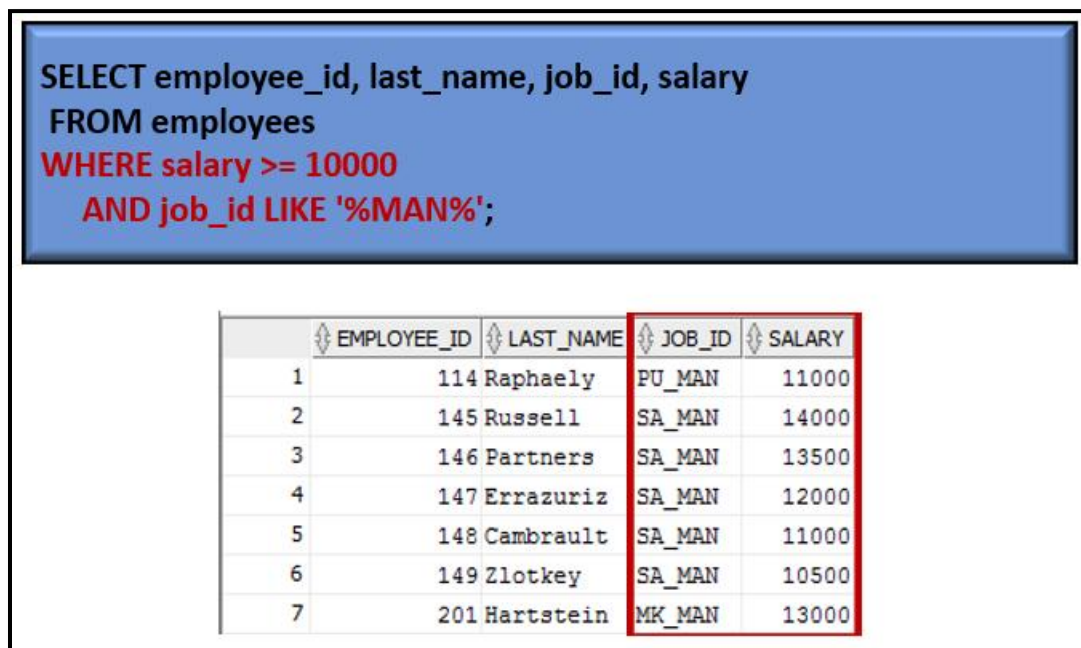
Imagen n°1

Fuente: Elaboración propia

## Operador AND

Requiere que todos los componentes de la condición sean verdaderos para mostrar la información.

En el ejemplo, la sentencia SELECT retorna las filas de los empleados cuyo salario sea mayor o igual a 10000 Y su identificación de trabajo contenga el valor MAN.



```
SELECT employee_id, last_name, job_id, salary
FROM employees
WHERE salary >= 10000
AND job_id LIKE '%MAN%';
```

|   | EMPLOYEE_ID | LAST_NAME | JOB_ID | SALARY |
|---|-------------|-----------|--------|--------|
| 1 | 114         | Raphaely  | PU_MAN | 11000  |
| 2 | 145         | Russell   | SA_MAN | 14000  |
| 3 | 146         | Partners  | SA_MAN | 13500  |
| 4 | 147         | Errazuriz | SA_MAN | 12000  |
| 5 | 148         | Cambrault | SA_MAN | 11000  |
| 6 | 149         | Zlotkey   | SA_MAN | 10500  |
| 7 | 201         | Hartstein | MK_MAN | 13000  |

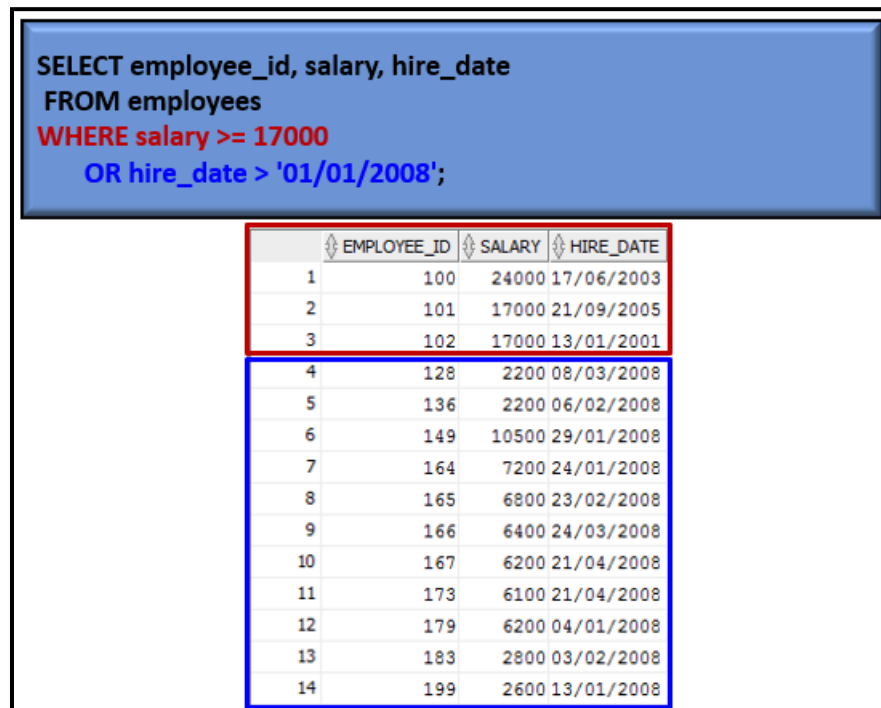
Imagen n°2

Fuente: Elaboración propia

## Operador OR

Requiere que alguno de los componentes de la condición sea verdadero para mostrar la información.

El ejemplo de la sentencia SELECT retorna las filas de los empleados cuyo salario sea mayor o igual a 17000 o su fecha de contratación sea mayor al 1 de enero del año 2008.



```
SELECT employee_id, salary, hire_date
FROM employees
WHERE salary >= 17000
OR hire_date > '01/01/2008';
```

| EMPLOYEE_ID | SALARY | HIRE_DATE        |
|-------------|--------|------------------|
| 1           | 100    | 24000 17/06/2003 |
| 2           | 101    | 17000 21/09/2005 |
| 3           | 102    | 17000 13/01/2001 |
| 4           | 128    | 2200 08/03/2008  |
| 5           | 136    | 2200 06/02/2008  |
| 6           | 149    | 10500 29/01/2008 |
| 7           | 164    | 7200 24/01/2008  |
| 8           | 165    | 6800 23/02/2008  |
| 9           | 166    | 6400 24/03/2008  |
| 10          | 167    | 6200 21/04/2008  |
| 11          | 173    | 6100 21/04/2008  |
| 12          | 179    | 6200 04/01/2008  |
| 13          | 183    | 2800 03/02/2008  |
| 14          | 199    | 2600 13/01/2008  |

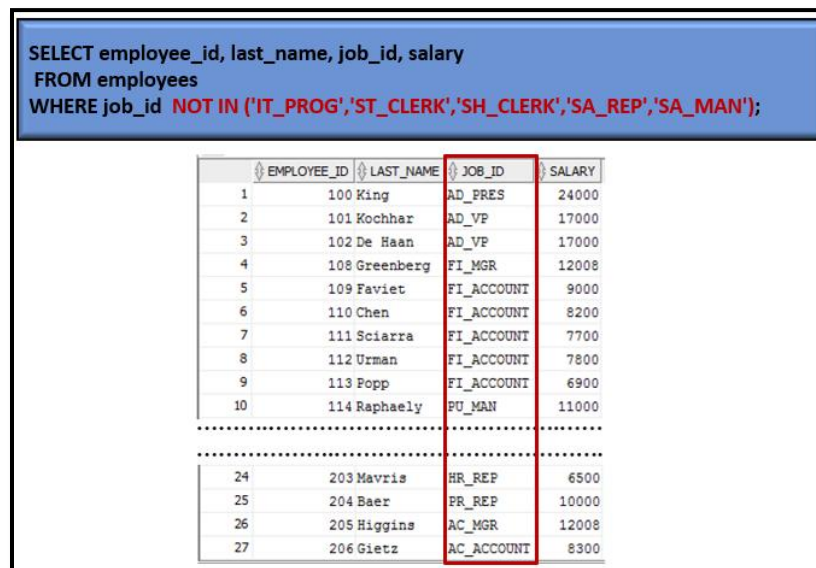
Imagen n°3  
Fuente: Elaboración propia



## Operador NOT

Invierte la lógica de la expresión que está a su derecha.

En el ejemplo, la sentencia muestra información de los empleados cuyo trabajo sea diferente de IT\_PROG, ST\_CLERK, SH\_CLERK, SA\_REP o SA\_MAN.



```
SELECT employee_id, last_name, job_id, salary
FROM employees
WHERE job_id NOT IN ('IT_PROG','ST_CLERK','SH_CLERK','SA_REP','SA_MAN');
```

| EMPLOYEE_ID | LAST_NAME     | JOB_ID     | SALARY |
|-------------|---------------|------------|--------|
| 1           | 100 King      | AD_PRES    | 24000  |
| 2           | 101 Kochhar   | AD_VP      | 17000  |
| 3           | 102 De Haan   | AD_VP      | 17000  |
| 4           | 108 Greenberg | FI_MGR     | 12008  |
| 5           | 109 Fawiet    | FI_ACCOUNT | 9000   |
| 6           | 110 Chen      | FI_ACCOUNT | 8200   |
| 7           | 111 Sciarra   | FI_ACCOUNT | 7700   |
| 8           | 112 Urman     | FI_ACCOUNT | 7800   |
| 9           | 113 Popp      | FI_ACCOUNT | 6900   |
| 10          | 114 Raphaely  | PU_MAN     | 11000  |
| 24          | 203 Mavris    | HR_REP     | 6500   |
| 25          | 204 Baer      | PR_REP     | 10000  |
| 26          | 205 Higgins   | AC_MGR     | 12008  |
| 27          | 206 Gietz     | AC_ACCOUNT | 8300   |

Imagen n°4

Fuente: Elaboración propia

## Reglas de precedencia para Operadores

Las reglas de precedencia determinan el orden en que las expresiones son evaluadas o calculadas. Si se desea modificar el orden de precedencia por defecto se debe utilizar paréntesis.

| ORDEN DE PRECEDENCIA | SIGNIFICADO                    |
|----------------------|--------------------------------|
| 1                    | Operadores Aritméticos         |
| 2                    | Operador de Concatenación      |
| 3                    | Condiciones de Comparaciones   |
| 4                    | IS [NOT], NULL, LIKE, [NOT] IN |
| 5                    | [NOT] BETWEEN                  |
| 6                    | No igual                       |
| 7                    | Condición lógica NOT           |
| 8                    | Condición lógica AND           |
| 9                    | Condición lógica OR            |

**Imagen n°5**  
**Fuente: Elaboración propia**

El ejemplo consta de dos condiciones:

- La primera condición que se ejecuta es que la identificación del trabajo sea AD\_PRES Y el salario sea mayor a 15000.
- La segunda condición que se ejecuta es que la identificación del trabajo sea SA\_REP.

Por lo tanto, la sentencia SELECT se ejecuta, por orden de precedencia de la siguiente manera: "Seleccione la fila si el empleado posee el trabajo con identificación AD\_PRES Y su salario es mayor que 15000, O si el empleado posee el trabajo SA\_REP".

*¿Cuál es la relevancia de usar paréntesis si se desea modificar el orden de precedencia por defecto?*



```

SELECT last_name, job_id, salary
FROM employees
WHERE job_id = 'SA_REP' } 2
    OR job_id = 'AD_PRES' } 1
    AND salary > 15000;

```

|       | LAST_NAME  | JOB_ID  | SALARY |
|-------|------------|---------|--------|
| 1     | King       | AD_PRES | 24000  |
| 2     | Tucker     | SA_REP  | 10000  |
| 3     | Bernstein  | SA_REP  | 9500   |
| 4     | Hall       | SA_REP  | 9000   |
| 5     | Olsen      | SA_REP  | 8000   |
| 6     | Cambrault  | SA_REP  | 7500   |
| 7     | Tuvault    | SA_REP  | 7000   |
| 8     | King       | SA_REP  | 10000  |
| 9     | Sully      | SA_REP  | 9500   |
| 10    | McEwen     | SA_REP  | 9000   |
| ..... |            |         |        |
| 28    | Taylor     | SA_REP  | 8600   |
| 29    | Livingston | SA_REP  | 8400   |
| 30    | Grant      | SA_REP  | 7000   |
| 31    | Johnson    | SA_REP  | 6200   |

Imagen n°6  
Fuente: Elaboración propia

El ejemplo consta de dos condiciones:

- La primera sentencia que se ejecuta es que la identificación del trabajo sea AD\_PRES o SA\_REP.
- La segunda condición que se ejecuta es que el salario sea mayor a 15000.

La sentencia SELECT se ejecuta, por orden de precedencia es de la siguiente manera : "Selecione la fila si el trabajo del empleado es AD\_PRES O SA\_REP Y si el empleado además posee un salario mayor que 15000".

```

SELECT last_name, job_id, salary
FROM employees
WHERE (job_id = 'SA_REP'
    OR job_id = 'AD_PRES') } 1
    AND salary > 15000; } 2

```

|   | LAST_NAME | JOB_ID  | SALARY |
|---|-----------|---------|--------|
| 1 | King      | AD_PRES | 24000  |

Imagen n°7  
Fuente: Elaboración propia

## Lenguaje de Manipulación de Datos (DML)

El DML (Lenguaje de Modificación de Datos) es una de las partes fundamentales del lenguaje SQL. Lo forman las instrucciones capaces de modificar (añadir, cambiar o eliminar) los datos de las tablas. Al conjunto de instrucciones DML que se ejecutan consecutivamente, se le llama transacción.

Se ejecuta una sentencia DML cuando:

- Se agregan filas nuevas a una tabla.
- Se modifican las filas existentes de una tabla.
- Se eliminan filas existentes de una tabla.

Al conjunto de instrucciones DML que se ejecutan consecutivamente, se las llama transacciones y se pueden anular todas ellas o aceptar, ya que una instrucción DML no es realmente efectuada en las tablas hasta que se confirma el cambio usando la sentencia COMMIT.

En todas estas consultas, el único dato devuelto por Oracle es el número de filas que se han modificado.

*¿Qué pasaría si deseas agregar nuevas filas a una tabla existente?*



## Sentencia INSERT

Para agregar nuevas filas a una tabla se debe usar la sentencia INSERT.

La nueva fila por insertar debe contener los valores para cada columna de la tabla.

Se deben enumerar ó insertar los valores en el orden por defecto de las columnas de la tabla.

Dado que la nueva fila a insertar en la tabla debe contener los valores para cada columna se puede omitir listar las *columnas* en la cláusula INSERT. En este caso, los valores a insertar deben ser listados de acuerdo con el orden que tengan las columnas en la tabla y se deben asignar todos los valores para las columnas. Los valores de caracteres y de fecha se deben escribir entre comillas simples.

Sintaxis:

```
INSERT INTO tabla [(columna [, columna...])]
VALUES (valor [, valor...]);
```

Con esta sintaxis se inserta una fila a la vez.

Imagen n°8

Fuente: Elaboración propia

Donde:

- *tabla*: es el nombre de la tabla en la cual se desean insertar las nuevas filas.
- *columna*: es el nombre de la columna en la tabla donde se desea ingresar el nuevo dato.
- *valor*: es el nuevo valor para la columna.

| DEPARTMENTS   |                 |            |             | Nueva Fila    |                  |            |             |
|---------------|-----------------|------------|-------------|---------------|------------------|------------|-------------|
| DEPARTMENT_ID | DEPARTMENT_NAME | MANAGER_ID | LOCATION_ID | DEPARTMENT_ID | DEPARTMENT_NAME  | MANAGER_ID | LOCATION_ID |
| 10            | Administration  | 200        | 1700        | 70            | Public Relations | 204        | 2700        |
| 20            | Marketing       | 201        | 1800        |               |                  |            |             |
| 30            | Purchasing      | 114        | 1700        |               |                  |            |             |
| 40            | Human Resources | 203        | 2400        |               |                  |            |             |
| 50            | Shipping        | 121        | 1500        |               |                  |            |             |
| 60            | IT              | 103        | 1400        |               |                  |            |             |
| 80            | Sales           | 145        | 2500        |               |                  |            |             |
| .....         | .....           | .....      | .....       |               |                  |            |             |
| .....         | .....           | .....      | .....       |               |                  |            |             |
| 250           | Retail Sales    | (null)     | 1700        |               |                  |            |             |
| 260           | Recruiting      | (null)     | 1700        |               |                  |            |             |
| 270           | Payroll         | (null)     | 1700        |               |                  |            |             |

| Nueva fila agregada en la tabla DEPARTMENTS |                  |            |             |
|---------------------------------------------|------------------|------------|-------------|
| DEPARTMENT_ID                               | DEPARTMENT_NAME  | MANAGER_ID | LOCATION_ID |
| 10                                          | Administration   | 200        | 1700        |
| 20                                          | Marketing        | 201        | 1800        |
| 30                                          | Purchasing       | 114        | 1700        |
| 40                                          | Human Resources  | 203        | 2400        |
| 50                                          | Shipping         | 121        | 1500        |
| 60                                          | IT               | 103        | 1400        |
| 70                                          | Public Relations | 204        | 2700        |
| 80                                          | Sales            | 145        | 2500        |
| .....                                       | .....            | .....      | .....       |
| .....                                       | .....            | .....      | .....       |
| 250                                         | Retail Sales     | (null)     | 1700        |
| 260                                         | Recruiting       | (null)     | 1700        |
| 270                                         | Payroll          | (null)     | 1700        |

**Imagen n°9**  
Fuente: Elaboración propia

La nueva fila por insertar debe contener los valores para cada columna de la tabla. Se deben enumerar o insertar los valores en el orden por defecto de las columnas de la tabla.

Dado que la nueva fila a insertar en la tabla debe contener los valores para cada columna se puede omitir listar las columnas en la cláusula INSERT. En este caso, los valores a insertar deben ser listados de acuerdo con el orden que tengan las columnas en la tabla y se deben asignar todos los valores para las columnas. Los valores de caracteres y de fecha se deben escribir entre comillas simples.

En el ejemplo, ambas sentencias insertan una nueva fila a la tabla DEPARTMENTS con el departamento 70. En la primera sentencia se indica en la cláusula INSERT cada columna de la tabla en donde se insertarán los nuevos valores especificados en la cláusula VALUES.

**Imagen n°10**  
Fuente: Elaboración propia

Ejemplo:

```
INSERT INTO departments(department_id, department_name,  
                        manager_id, location_id)  
VALUES (70, 'Public Relations', 100, 1700);
```

```
INSERT INTO departments  
VALUES (70, 'Public Relations', 100, 1700);
```

En la segunda sentencia, sólo se indican los valores a insertar, por lo tanto, se insertarán en el orden por defecto que las columnas tengan en la tabla.

Para insertar valores Nulos en las columnas se pueden realizar de forma implícita y explícita. En el método implícito se omite la columna en la cláusula INSERT. A través de la forma explícito se debe especificar la palabra NULL en la cláusula VALUES.

La función de Oracle SYSDATE almacena la fecha y hora actual de la Base de Datos.

Los errores comunes que se pueden presentar al momento de insertar nuevas filas en una tabla son:

- Que un valor Mandatorio (obligatorio) para una columna de la tabla definida como NOT NULL se haya omitido en la sentencia INSERT.
- Violación por valores duplicados para columnas definidas como únicas (fundamentalmente para las columnas de Claves Primarias).
- Violación de Integridad Referencial o de Claves Foráneas.
- El tipo de dato del valor que se va a insertar no coincide con el tipo de dato definido para la columna de la tabla correspondiente.
- El valor que se está insertando es demasiado largo para la columna sobre la cual se desea insertar el valor.

*¿Cuál es la razón que explica que los valores de caracteres y de fecha se deban escribir entre comillas simples?*



### Ejemplo: Inserción valores Nulos

Implicítamente

```
INSERT INTO departments  
(department_id, department_name)  
VALUES (30, 'Purchasing');
```

Explícitamente

```
INSERT INTO departments  
VALUES (100, 'Finance', NULL, NULL);
```

### Ejemplo: Inserción valores Especiales

```
INSERT INTO employees (employee_id, first_name, last_name, email,  
phone_number, hire_date, job_id,  
salary, commission_pct, manager_id,  
department_id)  
VALUES (113, 'Louis', 'Popp', 'LPOPP',  
'515.124.4567', SYSDATE, 'AC_ACCOUNT',  
6900, NULL, 205, 100);
```

Imagen n°11  
Fuente: Elaboración propia

## Objeto Secuencia

Una secuencia sirve para generar automáticamente números distintos.

Se pueden utilizar para generar valores para columnas que se utilizan como clave forzada (claves cuyo valor no interesa, sólo sirven para identificar las filas de una tabla) o para cualquier columna que corresponde a un valor correlativo.

Es una rutina interna de Oracle la que realiza la función de generar un número distinto cada vez.

Las secuencias se almacenan independientemente de la tabla, por lo que la misma secuencia se puede utilizar para diversas tablas.

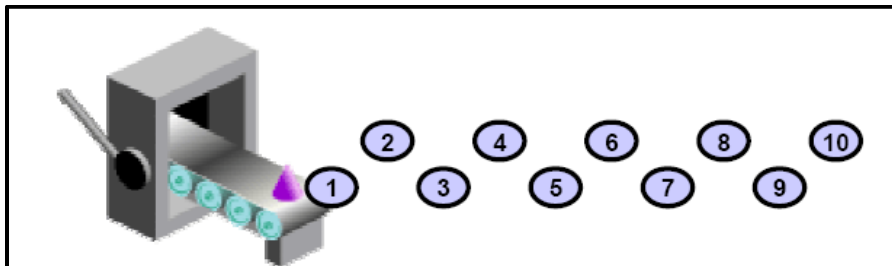


Imagen n°12  
Fuente: Elaboración propia



**Sintaxis:**

```
CREATE SEQUENCE secuencia  
[INCREMENT BY n]  
[START WITH n]  
[MAXVALUE n]  
[MINVALUE n];
```

**Imagen n°13****Fuente: Elaboración propia****Donde:**

- **secuencia:** es el nombre que se le da al objeto de secuencia a crear.
- **INCREMENT BY:** indica cuánto se incrementa la secuencia cada vez que se usa. Por defecto se incrementa de uno en uno.
- **START WITH:** indica el valor inicial de la secuencia (por defecto 1).
- **MAXVALUE:** máximo valor que puede tomar la secuencia. Por defecto es hasta el 10 elevado a 27.
- **MINVALUE:** mínimo valor que puede tomar la secuencia. Por defecto -10 elevado a 26. MINVALUE debe ser mayor o igual que o igual a START WITH, y debe ser menor a MAXVALUE.

Ejemplo: Se crea una secuencia con nombre DEPT\_DEPTID\_SEQ para ser usada como Clave Primaria de la tabla DEPARTMENTS.

**Ejemplo:**

```
CREATE SEQUENCE dept_deptid_seq  
INCREMENT BY 10  
START WITH 120  
MAXVALUE 9999  
NOCICLE;
```

**Imagen n°14****Fuente: Elaboración propia**

## Uso de Objeto Secuencia

- NEXTVAL: retorna el siguiente valor disponible de la secuencia. Esta retorna un valor único cada vez que es referenciado, por lo tanto es diferente para cada usuario que utilice la secuencia.
- CURRVAL: obtiene el valor actual de la secuencia.

Ambas pseudocolumnas pueden ser utilizadas en:

- Una consulta SELECT que no lleve DISTINCT, ni grupos, ni sea parte de una vista, ni sea subconsulta de otro SELECT, UPDATE o DELETE.
- Una subconsulta SELECT en una instrucción INSERT.
- La cláusula VALUES de la instrucción INSERT.
- La cláusula SET de la instrucción UPDATE
- No se puede utilizar como valor para la cláusula DEFAULT de un campo de tabla. Su uso más habitual es como apoyo al comando INSERT.

En el primer ejemplo, se obtiene el valor de la secuencia.

Ejemplo: para saber el valor de la secuencia

```
SELECT dept_deptid_seq.NEXTVAL  
FROM dual;
```

Imagen n°15

Fuente: Elaboración propia

En el segundo ejemplo, se inserta un nuevo departamento con el nombre Support y la identificación de este nuevo departamento será el siguiente valor de la secuencia dept\_deptid\_seq.

Ejemplo: para insertar una fila

```
INSERT INTO departments(department_id, department_name, location_id)  
VALUES (dept_deptid_seq.NEXTVAL, 'Support', 2500);
```

Imagen n°16

Fuente: Elaboración propia

Ejemplo: para eliminar una secuencia

```
DROP SEQUENCE dept_deptid_seq;
```

Imagen n°17  
Fuente: Elaboración propia

## Sentencia UPDATE

Para modificar las filas existentes en una tabla se debe utilizar la sentencia UPDATE.

Se puede actualizar más de una fila y/o una columna (si es necesario) con la sentencia UPDATE.

Imagen n°18  
Fuente: Elaboración propia

Sintaxis:

```
UPDATE tabla  
SET      columna = valor [, columna = valor, ...]  
[WHERE condición];
```

Se puede actualizar más de una fila y/o una columna (si es necesario).  
Si se omite la cláusula WHERE se actualizan todas las filas.

Donde:

- tabla: es el nombre de la tabla que se desea actualiza.
- columna: es el nombre de la(s) columna(s) de la tabla que se desea actualizar.
- valor: es el valor con el que se actualizará la columna.
- condición: permite identificar las filas que se desean actualizar.

| EMPLOYEES   |           |               |
|-------------|-----------|---------------|
| EMPLOYEE_ID | LAST_NAME | DEPARTMENT_ID |
| 100         | King      | 90            |
| 101         | Kochhar   | 90            |
| 102         | De Haan   | 90            |
| 103         | Hunold    | 60            |
| 104         | Ernst     | 60            |
| 203         | Mavris    | 40            |
| 204         | Baer      | 70            |
| 205         | Higgins   | 110           |
| 206         | Gietz     | 110           |

Actualizar filas en la tabla EMPLOYEES

| EMPLOYEE_ID | LAST_NAME | DEPARTMENT_ID |
|-------------|-----------|---------------|
| 100         | King      | 90            |
| 101         | Kochhar   | 90            |
| 102         | De Haan   | 90            |
| 103         | Hunold    | 110           |
| 104         | Ernst     | 110           |
| 203         | Mavris    | 40            |
| 204         | Baer      | 70            |
| 205         | Higgins   | 110           |
| 206         | Gietz     | 110           |

Imagen n°19  
Fuente: Elaboración propia

Ejemplo: En el primer ejemplo, la sentencia actualiza el salario actual de los empleados en 25% más del valor que posee. La actualización se efectuará para los empleados cuyo salario sea menos a 5000.

Ejemplo: Actualización de filas que cumplan con una condición

```
UPDATE employees
SET salary = salary * 1.25
WHERE salary < 5000;
```

Ejemplo: Actualización de todas las filas de la tabla

```
UPDATE employees
SET hire_date = SYSDATE - 2,
    salary = salary - (salary * 0.085);
```

Imagen n°20  
Fuente: Elaboración propia

En el segundo ejemplo, la sentencia actualiza la fecha de contrato y el salario de TODOS los empleados. La fecha de contrato se actualiza en dos días menos a la fecha del sistema y el salario en 8,5% menos del salario actual.

## Sentencia DELETE

Para eliminar las filas existentes en una tabla se debe utilizar la sentencia DELETE.

Imagen n°21

Fuente: Elaboración propia

Sintaxis:

```
DELETE [FROM] tabla
[WHERE condición];
```

Si se desean eliminar algunas filas de la tabla se debe incluir la cláusula WHERE.

Donde:

- tabla: es la tabla que se desea actualizar.
- condición: permite identificar las columnas que se desean eliminar.

| DEPARTMENT_ID | DEPARTMENT_NAME | MANAGER_ID | LOCATION_ID |
|---------------|-----------------|------------|-------------|
| 10            | Administration  | 200        | 1700        |
| 20            | Marketing       | 201        | 1800        |
| 30            | Purchasing      | 114        | 1700        |
| 40            | Human Resources | 203        | 2400        |
| 50            | Shipping        | 121        | 1500        |
| 60            | IT              | 103        | 1400        |
| .....         |                 |            |             |
| 250           | Retail Sales    | (null)     | 1700        |
| 260           | Recruiting      | (null)     | 1700        |
| 270           | Payroll         | (null)     | 1700        |

**Elimina una fila desde tabla DEPARTMENTS:**

| DEPARTMENT_ID | DEPARTMENT_NAME | MANAGER_ID | LOCATION_ID |
|---------------|-----------------|------------|-------------|
| 10            | Administration  | 200        | 1700        |
| 20            | Marketing       | 201        | 1800        |
| 30            | Purchasing      | 114        | 1700        |
| 40            | Human Resources | 203        | 2400        |
| 50            | Shipping        | 121        | 1500        |
| .....         |                 |            |             |
| .....         |                 |            |             |

Imagen n°22

Fuente: Elaboración propia

Eliminando todas las filas de la tabla: si se omite la cláusula WHERE se eliminan todas las filas de la tabla. En el ejemplo, se Elimina todas las filas de la tabla EMPLOYEES.

Ejemplo: eliminando todas las filas de la tabla.

```
DELETE employees;
```

Imagen n°23

Fuente: Elaboración propia

Eliminando filas específicas: Para eliminar filas específicas en una tabla se debe incluir la cláusula WHERE.

- En el primer ejemplo, se elimina desde tabla DEPARTMENTS la(s) fila(s) cuyo nombre de departamento sea Finance.
- En el segundo ejemplo, se elimina desde tabla DEPARTMENTS las filas cuya identificación de departamento sea 30 ó 40.
- En el último ejemplo, se elimina desde tabla EMPLOYEES las filas de los empleados cuyo salario esté entre los 2000 y 5000.

Ejemplo: eliminando filas específicas de la tabla.

```
DELETE FROM departments  
WHERE department_name = 'Finance';
```

```
DELETE FROM departments  
WHERE department_id IN(30, 40);
```

```
DELETE employees  
WHERE salary between 2000 AND 5000;
```

Imagen n°24

Fuente: Elaboración propia

## Sentencia TRUNCATE

Una forma más eficiente de limpiar una tabla (eliminar el contenido de toda la tabla) es a través de la sentencia TRUNCATE.

Imagen n°25  
Fuente: Elaboración propia

Sintaxis:

```
TRUNCATE TABLE nombre_tabla;
```

La sentencia TRUNCATE (DDL) es más rápida que la sentencia DELETE (DML) pero más difícil de “deshacer” los cambios, pues TRUNCATE tiene un COMMIT implícito.

En el ejemplo, se eliminan todas las filas de la tabla EMPLOYEES.

Ejemplo

```
TRUNCATE TABLE employees;
```

Imagen n°26  
Fuente: Elaboración propia

*¿Cuál es la mayor dificultad que posee la utilización de la sentencia Delete?*



# Conclusión

En el siguiente Material de Estudio, se conocieron los operadores lógicos: como el AND, OR y NOT.

Además de las sentencias INSERT, que permiten añadir nuevas filas a una tabla. La sentencia UPDATE, que permite la modificación o actualización de datos dentro de una tabla. Y el comando DELETE, que elimina filas de una tabla de acuerdo con una condición o incluso todos los registros de ella.

Para ello, se utilizaron varios ejemplos, detallando la lógica y resultado esperado.



# Referencias

Parte de la información ha sido extraída desde Oracle Academy. (s.f.).  
(<https://myacademy.oracle.com>)



**4** INSTITUCION  
**ACREDITADA**  
NIVEL AVANZADO  
AÑOS Hasta octubre 2025



GESTIÓN INSTITUCIONAL Y DOCENCIA DE PREGRADO